

Problem Set #5

Due date: October 25, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- **You are not allowed to use material that has not been covered in class.**
 - The only string methods you are allowed to use are: split, strip and join
 - The only list methods you are allowed to use are: append, pop and del
 - The only dictionary methods you are allowed to use are: get, pop and del
 - You can use the len function and slicing
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS5.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 5 channel in Microsoft Teams.

The main focus of this problem set is to practice dictionary syntax and efficiently solve problems using dictionaries. Part of your design for these questions comes from the proper use of dictionaries to solve the problem.

Note that most problems require you to create a dictionary, and in some problems, the output from other functions can be used to solve other problems, creating a cleaner solution as you reuse code, which is also part of your design grade.

Take a moment to outline the process of your algorithm before you start coding, so you can identify where to reuse code from other functions and avoid repeated code!



To receive any credit for any of these problems, you are not allowed to use the string methods `lower()` or `upper()`, you must define your own helper functions outside the function definition. You are not allowed to use any built-in functions or libraries to count.

Part 1: Warm-up

Problem 1.1 (10 pts) Write the Python code to implement the function `invert_dict` that takes a dictionary and creates a new dictionary that is the inverted version of the original one. That is, for each key-value pair in the input dictionary, make the value a key and the key a value. The values in the new dictionary will be stored in a list. Assume that all keys in the input dictionary are immutable.

Example:

```
>>> invert_dict({'a': 1, 'b': 2, 'c': 1, 'd': 2, 3.75: 7})
{1: ['a', 'c'], 2: ['b', 'd'], 7: [3.75]}
```

Problem 1.2 (10 pts) Write the Python code to implement the function `char_frequency` that takes in a string as an input and returns the alphabet character frequency dictionary. Non-alphabet characters should be ignored, and both uppercase and lowercase letters should be treated the same. The key in the dictionary must be all lowercase letters.

Example:

```
>>> char_frequency("~ABC abc!!!!!! !!!!!! ")
{'a': 2, 'b': 2, 'c': 2}
```

Problem 1.3 (10 pts) Write the Python code to implement the function `frequency_difference` that takes two strings and returns a dictionary containing characters that appear in the first string but not in the second, along with their frequencies. Both uppercase and lowercase letters should be treated the same. The returned dictionary should represent the "extra" characters in the first string compared to the second.

Example:

```
>>> frequency_difference("abcd!!", "A~")
{'b': 1, 'c': 1, 'd': 1, '!': 2}
```

Problem 1.4 (10 pts) Write the Python code to implement the function `find_missing_pangram_chars` that takes a string and returns a list of missing characters needed to make the string a pangram (a sentence containing every letter of the alphabet at least once).

Example:

```
>>> find_missing_pangram_chars("The quick brown fox")
['a', 'd', 'g', 'j', 'l', 'm', 'p', 's', 'v', 'y', 'z']
```

Part 2: Fun with Anagrams!

Functions from previous questions are very useful here!

Problem 2.1 (10 pts) An anagram is a word or phrase that's formed by rearranging the letters of another word or phrase. Write the Python code to implement the function `are_anagrams` that takes two strings and returns True if they are anagrams of each other, otherwise return False. Ignore spaces and case differences.

Examples:

```
>>> are_anagrams("Listen", "Silent")
True
>>> are_anagrams("Listen", "Google")
False
```

Problem 2.2 (10 pts) Write the Python code to implement the function `find_anagrams`, that takes a string and a list of words and returns a list of words that are anagrams of the given string. Ignore spaces and case differences.

Examples:

```
>>> find_anagrams("listen", ["ENlist", "google", "inLets", "banana"])
['ENlist', 'inLets']
```

Problem 2.3 (10 pts) Write the Python code to implement the function `find_anagram_pairs` that takes a list of strings and returns a list of all pairs of indexes where the two strings are anagrams of each other. Ignore spaces and case differences.

Examples:

```
>>> find_anagram_pairs(["listen", "silent", "enlist", "inlets", "google"])
[(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]
```